



FpAuthAnalysis

Fingerprint Authentication Misuse Analyzer

A Static Analysis Framework Inspired by NDSS Research

Course Project Report

Submitted to: Dr. Ramesh Babu Bathula

Course Name: Computer and Network Security

Course Code: 22CST352

Ajay Vanama – 2023UCP1625

Jagat Singh – 2023UCP1616

Department of Computer Science
MNIT

May 10, 2026

Contents

Abstract	2
Introduction	3
0.1 Background	3
0.2 Problem Statement	3
0.3 Project Objectives	3
1 Related Work	4
1.1 NDSS Research Paper	4
1.2 Existing Android Security Tools	4
2 System Architecture	5
2.1 Architecture Overview	5
2.2 Workflow Pipeline	6
3 Implementation	7
3.1 Technology Stack	7
3.2 Directory Structure	7
3.3 APK Decompilation	7
3.4 Permission Analysis	7
3.5 Fingerprint API Detection	8
3.6 Lifecycle Analysis	8
4 Experimental Evaluation	9
4.1 Dataset	9
4.2 Experimental Setup	9
4.3 Permission Detection Results	9
5 Results and Discussion	10
5.1 Observations	10
5.2 Limitations	10
Conclusion	10
References	11

Abstract

This project presents **FpAuthAnalysis**, a lightweight static analysis framework designed to detect fingerprint authentication misuses in Android applications. Inspired by the NDSS research paper *An Empirical Study on Fingerprint API Misuse with Lifecycle Analysis in Real-world Android Apps*, the system analyzes Android APKs for insecure biometric implementations.

The framework performs APK decompilation, AndroidManifest permission analysis, fingerprint API detection, lifecycle misuse analysis, callback verification, and cryptographic binding checks. The tool identifies vulnerabilities such as missing CryptoObject validation, improper lifecycle cancellation, and insecure authentication callbacks.

The system uses static analysis only and does not require root access or runtime instrumentation. Experimental evaluation on a dataset of Android applications demonstrates successful detection of biometric authentication implementations and potential misuse patterns.

Keywords: Android Security, Fingerprint Authentication, BiometricPrompt, FlowDroid, APK Analysis, Static Analysis, Lifecycle Analysis, Android Reverse Engineering, CryptoObject, NDSS

Introduction

0.1 Background

Biometric authentication has become widely adopted in Android applications due to convenience and improved user experience. Android provides APIs such as:

- FingerprintManager
- BiometricPrompt
- CryptoObject

However, incorrect implementation of these APIs can lead to severe security vulnerabilities including authentication bypass and unauthorized access.

0.2 Problem Statement

Many Android applications misuse fingerprint authentication APIs by:

- Missing cryptographic validation
- Improper lifecycle handling
- Weak callback verification
- Authentication without CryptoObject binding

These vulnerabilities can compromise application security even when biometric authentication is enabled.

0.3 Project Objectives

1. Collect and analyze Android APKs using static analysis.
2. Detect biometric permissions in AndroidManifest.xml.
3. Identify usage of fingerprint authentication APIs.
4. Detect lifecycle-related biometric vulnerabilities.
5. Analyze authentication callbacks.
6. Verify secure CryptoObject usage.
7. Generate structured vulnerability reports.

Related Work

1.1 NDSS Research Paper

The project is inspired by the NDSS paper:

An Empirical Study on Fingerprint API Misuse with Lifecycle Analysis in Real-world Android Apps

The paper analyzed more than 1300 Android applications and identified multiple categories of fingerprint authentication vulnerabilities.

1.2 Existing Android Security Tools

- FlowDroid
- Androguard
- JADX
- APKTool

These tools provide static analysis support but do not directly classify fingerprint authentication misuse.

System Architecture

2.1 Architecture Overview

3.1 Architecture Overview

The following figure illustrates the high-level architecture of FpAuthAnalysis and the flow of data through each component.

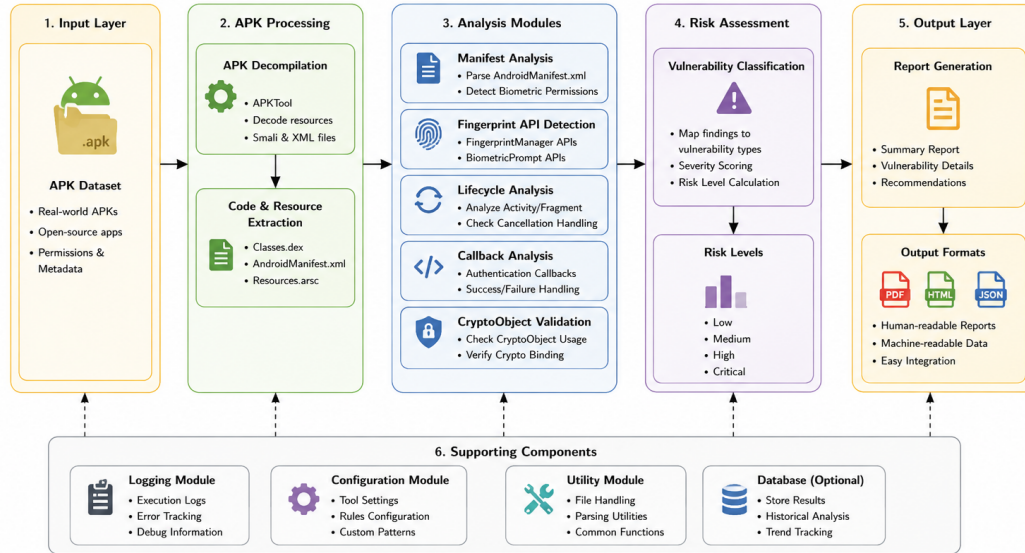
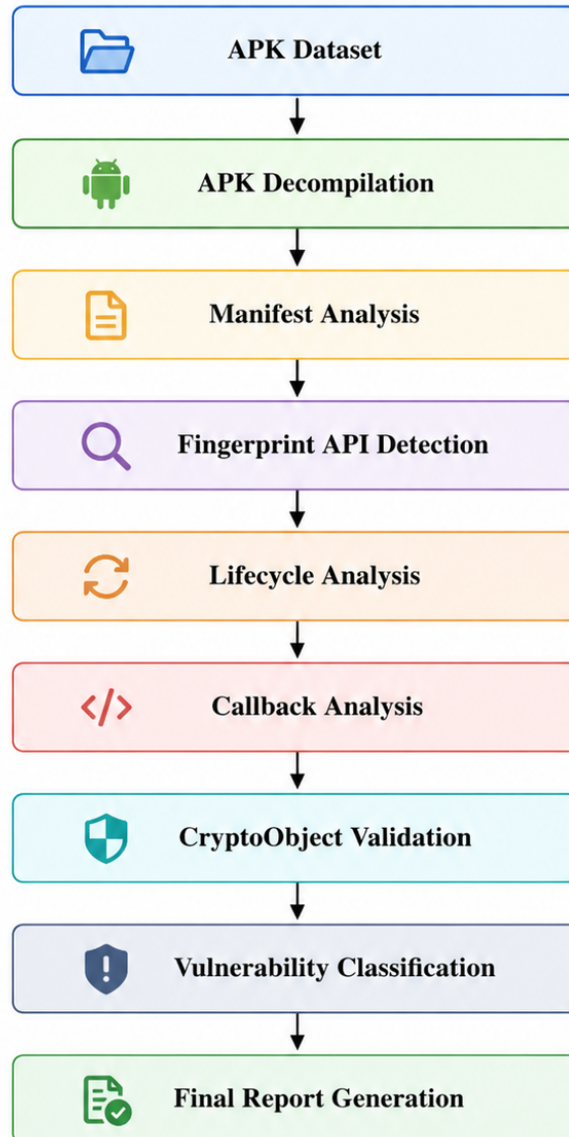


Figure 1: Architecture Overview of FpAuthAnalysis

2.2 Workflow Pipeline

3.2 Workflow Pipeline



Implementation

3.1 Technology Stack

- Python 3.12
- Java 11
- APKTool
- Androguard
- FlowDroid
- Android SDK

3.2 Directory Structure

FpAuthAnalysis/

```
scripts/  
results/  
apks/  
decoded_apks/  
README.md  
requirements.txt  
.gitignore
```

3.3 APK Decompilation

APKTool was used to decode Android APKs:

```
apktool d app.apk -o output_folder
```

3.4 Permission Analysis

The system scans AndroidManifest.xml for:

- android.permission.USE_F*INGERPRINT*
- android.permission.USE_B*IOMETRIC*

3.5 Fingerprint API Detection

The framework detects:

```
FingerprintManager.authenticate()  
BiometricPrompt.authenticate()
```

3.6 Lifecycle Analysis

Lifecycle methods analyzed:

- onPause()
- onStop()
- onResume()
- onDestroy()

The system checks whether biometric authentication is properly cancelled during activity transitions.

Experimental Evaluation

4.1 Dataset

The dataset contains Android applications including:

- Dashlane
- Bitwarden
- KeePassDX
- Aegis Authenticator
- Standard Notes
- AppLock

4.2 Experimental Setup

- Ubuntu 24.04
- Java 11
- Python 3.12
- Android SDK Platform APIs

4.3 Permission Detection Results

Application	Fingerprint Permission Detected
Dashlane	Yes
Bitwarden	Yes
KeePassDX	Yes
Aegis	Yes
Standard Notes	Yes
AppLock	Yes

Table 4.1: Fingerprint Permission Analysis

Results and Discussion

5.1 Observations

The analysis revealed that several applications request biometric permissions and use fingerprint authentication APIs.

The framework successfully detected:

- Fingerprint permissions
- Biometric API usage
- Lifecycle handling patterns
- Authentication callbacks

5.2 Limitations

- Static analysis cannot observe runtime behavior.
- Heavy code obfuscation affects detection accuracy.
- Some APK bundles are difficult to analyze.

Conclusion

FpAuthAnalysis successfully demonstrates a static analysis approach for identifying fingerprint authentication misuses in Android applications.

The framework provides:

- Automated APK analysis
- Fingerprint API detection
- Lifecycle misuse analysis
- Vulnerability classification

Future work includes:

- Dynamic runtime analysis
- Frida-based instrumentation
- Machine learning classification

References

1. NDSS Research Paper on Fingerprint API Misuse
2. Android Developer Documentation
3. FlowDroid Documentation
4. Androguard Documentation
5. APKTool Documentation